Java OOP Laboratory Exercise: Campus Café Ordering System

ITS 202: Object Oriented Programming

Submitted by:

Mae-an S. ACABAL

Jella T. Quibot

Section: C

NOVEMBER 10, 2026

Concept Explanation

1. **if Statement:** We used if statements in Pastry.computePrice() and Meal.computePrice() to add extra charges only when needed, such as when a pastry is warmed or a meal is upsized or given extra sides. We also used them in Order to check whether an item slot is empty before processing it.

2. **if-else Statement:** We used if-else statements when the program had to choose between two outcomes, such as rejecting negative prices in setBasePrice(), checking the size of a beverage, limiting the number of items in an order, and applying a student discount or a voucher.

3. **Classes and Objects:** Our program has seven classes, each placed in its own Java file. We created the customer, menu item, and order objects in the main class, where we also demonstrated how the parts of the ordering system work together.

4. **Attributes and Behaviors:** Attributes store information about each object, such as a beverage's size or whether a pastry is warmed. Behaviors are the methods that define what an object can do, like calculating its price with computePrice() or adding an item with addItem().

5. **Object Interaction (Order):** The Order class connects a customer with up to three menu items. When printReceipt() is called, it gets the details and prices of the ordered items, computes the subtotal and total, and updates the customer's loyalty points.

6. **Java Naming Conventions:** We followed PascalCase for class names such as MenuItem and camelCase for methods and variables such as basePrice and computeTotal().

7. **Default Constructor:** Default constructors give objects starting values when no information is provided, for example a menu item with a placeholder name and a base price of 0.0.

8. **Parameterized Constructor:** Parameterized constructors set an object's information at the moment it is created. In the subclasses (Beverage, Pastry, Meal), we used super(...) to pass the item's name, price, and code to the parent MenuItem class.

9. **Constructor Overloading:** The Customer class has multiple constructors so a customer can be created with no information, with only a name, or with both a name and student status.

10. **public / private / protected:** Most instance variables are private to protect the data, while name and basePrice are protected so that the subclasses can use them. Methods that other classes need to call are public.

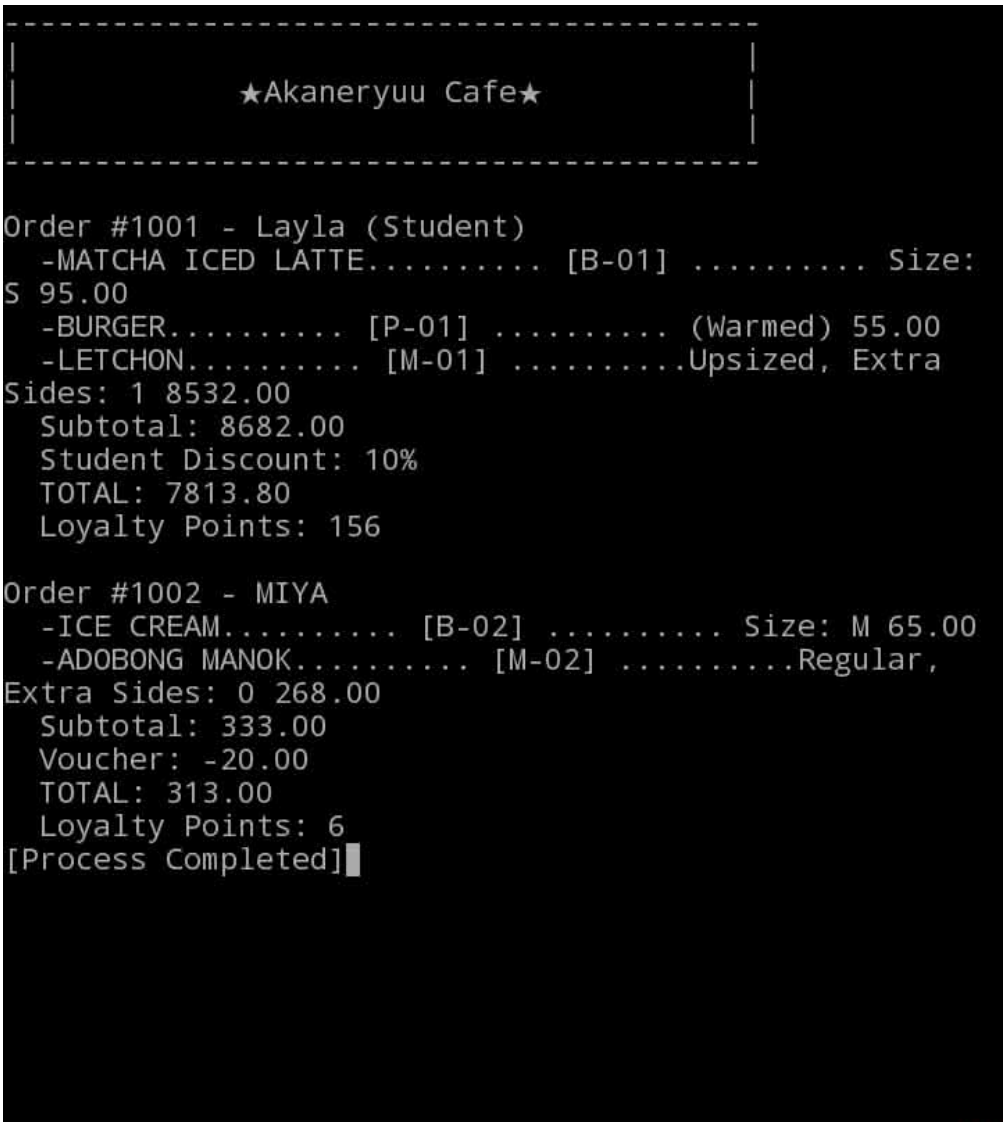
11. **Getters, Setters, and Data Hiding:** We used getters and setters instead of making variables public. Some setters include validation to prevent invalid values, such as negative prices or negative loyalty points.

12. **Method Overloading:** We created methods with the same name but different parameters, such as addPoints(int) and addPoints(double), computeTotal() and computeTotal(double), and printReceipt() and printReceipt(double) for orders with a voucher.

13. **Method Overriding:** Beverage, Pastry, and Meal each provide their own computePrice() and getDescription() instead of using the version inherited from MenuItem. We added the @Override annotation to these methods.

14. **Abstraction:** MenuItem is an abstract class because it is only the parent of the different menu items. Its abstract method computePrice() forces each subclass to define its own way of calculating the price.

Screenshot / Output:



The output shows two orders at Akaneryuu Cafe. Order #1001 (Layla, a student) has a 10% student discount applied to the subtotal and earns loyalty points. Order #1002 (Miya) uses a 20-peso voucher instead of a discount.

Challenges Encountered:

• **Comparing Strings:** We first used == to compare drink sizes, which compares references instead of content. We fixed this by using .equals() for each size and joining the conditions with ||.

• **Size Left Empty:** When an invalid size was passed to the Beverage constructor, the setter rejected it and left the size null, which could cause an error in computePrice(). We fixed this by assigning a default size of "S" before checking the given input.

• **Abstract Method:** At first we forgot to declare MenuItem as an abstract class and gave computePrice() a method body. We corrected this by adding the abstract keyword and leaving only the method declaration.

• **Points in Customer:** Our addPoints() methods originally replaced the existing points instead of adding to them. We fixed this by validating the incoming value first, using += to add points, and converting the amount spent into whole points at one point per 50 pesos.

• **Using += and *= Correctly:** We had trouble applying operators when computing the subtotal and total in Order. We fixed this by using variables to track the subtotal and total, updating them during the calculations, and returning the correct values at the end of the methods.